

21 — Relationale Datenbanken

Tabellen, Schlüssel, Beziehungen, Datentypen, ACID

HTW Berlin – Wirtschaftsingenieurwesen

SoSe 2026

Agenda

1. Warum verknüpfte Tabellen statt einer Riesentabelle?
2. Anatomie einer Tabelle: Zeilen, Spalten, Werte
3. Primärschlüssel – jede Zeile eindeutig
4. NULL – der Wert, der fehlt
5. Fremdschlüssel – Tabellen verbinden
6. Beziehungstypen 1:1, 1:n, n:m
7. SQL-Datentypen – die richtige Schublade
8. Transaktionen & ACID – die Sicherheitsgarantien

Lernziel: Das relationale Modell verstehen – damit wir in NB 22 mit SQLite *darauf* arbeiten können.

Wie wir heute arbeiten: Konzept → kurze *Live-Demo* (Tabellen mit Python-Mitteln nachgebaut) → *Selbst überlegen* (Denkaufgaben im Notebook 21).

Eine Riesentabelle? Lieber nicht.

Alles in einer Tabelle

bestell_id	kunde_name	kunde_email	produkt	preis	datum
10	Anna Müller	anna@example.de	Eco-Sneaker	89.95	2026-05-03
11	Anna Müller	anna@example.de	Hemp-High	109.00	2026-05-09

- Annas Mail steht **doppelt** – ändert sie sich, müssen *alle* Zeilen angefasst werden
- Tippfehler in einer Zeile → zwei Versionen von „Anna Müller“
- Ändert der Eco-Sneaker den Preis, stimmen alte Bestellungen plötzlich nicht mehr

Idee des relationalen Modells: jede „Sache“ einmal in ihrer eigenen Tabelle – und über **Schlüssel** verbinden.

Anatomie einer Tabelle

Anatomie einer Tabelle · kunde

Primärschlüssel (PK)
eindeutig, nie NULL, unveränderlich

NULL $\neq$ 0 $\neq$ "" — Wert fehlt / unbekannt
(prüfen mit IS NULL)

INTEGER	TEXT	TEXT	TEXT	BOOLEAN
kunde_id	vorname	nachname	email	newsletter
1	Anna	Müller	anna@example.de	1
2	Max	Schmidt	max@example.de	0
3	Lena	Weber	NULL	1

Zeile = Datensatz / Tupel
(genau eine Kundin)

Spalte = Attribut
(ein fester Datentyp)

- **Zeile** (Datensatz / Tupel) = genau ein Ding · **Spalte** (Attribut) = eine Eigenschaft mit *einem* festen Datentyp
- keine zwei identischen Zeilen · eindeutige Spaltennamen

Primärschlüssel - jede Zeile eindeutig

```
CREATE TABLE kunde (  
    kunde_id INTEGER PRIMARY KEY,    -- der Primärschlüssel  
    vorname TEXT,  
    nachname TEXT,  
    email TEXT  
);
```

Ein Primärschlüssel ist eine Spalte (oder Kombination), die jede Zeile **eindeutig identifiziert**:

- **eindeutig** – kein Wert kommt zweimal vor
- **nie NULL** – jede Zeile *muss* einen haben
- **unveränderlich** – ändert sich nach dem Anlegen nicht

Gute Kandidaten: Matrikelnummer, ISBN, oder eine automatisch hochgezählte ID. Schlechte: Name, E-Mail (ändern sich, sind nicht garantiert eindeutig)

NULL - der Wert, der fehlt

kunde_id	name	email
1	Anna Müller	anna@example.de
2	Max Schmidt	max@example.de
3	Lena Weber	NULL

NULL = hier steht (noch) kein Wert
nicht 0 · nicht leerer Text · nicht False
Abfrage: WHERE email IS NULL
NICHT WHERE email = NULL (findet nichts!)

- mögliche Bedeutungen: *unbekannt* · *nicht anwendbar* · *noch nicht eingetragen* – in Python: None
- **nie** beim Primärschlüssel; mit NOT NULL erzwingt man „diese Spalte ist Pflicht“

Faustregel: NULL heißt *kein Wert* – nicht 0, nicht "". Darum in Abfragen mit IS NULL prüfen, nie mit = NULL (mehr dazu in NB 23).

Fremdschlüssel - Tabellen verbinden

Ein **Fremdschlüssel** ist eine Spalte, die auf den **Primärschlüssel** einer anderen **Tabelle** zeigt.

```
CREATE TABLE bestellung (  
    bestell_id INTEGER PRIMARY KEY,  
    kunde_id   INTEGER,           -- Fremdschlüssel  
    FOREIGN KEY (kunde_id) REFERENCES kunde(kunde_id)  
);
```

- verbindet bestellung mit kunde – ohne Annas Daten zu kopieren
- **referentielle Integrität**: das DBMS lässt keinen FK zu, der ins Leere zeigt – keine „verwaisten“ Bestellungen

► **Live-Demo** – 02_fremdschluesssel_beziehung.py

Drei Beziehungstypen

1 : 1

jede Bestellung hat genau eine Rechnung



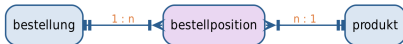
1 : n

ein Kunde, viele Bestellungen — FK auf der „n“-Seite



n : m

aufgelöst über bestellposition (zwei FKs)



Der **Fremdschlüssel steht auf der „n“-Seite**. Eine **n:m**-Beziehung braucht eine **Zwischentabelle** mit zwei Fremdschlüsseln.

📌 **Selbst überlegen** – Notebook 21, Übung 2.1: Beziehungen für ein Shop-System modellieren.

SQL-Datentypen - die richtige Schublade

Kategorie	Typen	wofür
Ganzzahl	INTEGER, BIGINT	IDs, Stückzahlen
Dezimal	DECIMAL(p, s)	Geldbeträge – exakt, keine Rundungsfehler
Gleitkomma	FLOAT, DOUBLE	wissenschaftliche Werte (nicht für Geld!)
Text	CHAR(n), VARCHAR(n), TEXT	feste / variable / lange Texte
Datum/Zeit	DATE, TIME, DATETIME, TIMESTAMP	Termine, Zeitstempel
weitere	BOOLEAN, ENUM, BLOB	Ja/Nein, feste Auswahl, Binärdaten

- richtige Wahl = Datenintegrität + Speichereffizienz + Performance

- Telefonnummern als **Text** (führende Nullen, +49, ...) nicht als Zahl

Transaktion - alles oder nichts

Eine **Transaktion** fasst mehrere Operationen zu einer **unteilbaren Einheit** zusammen.

BEGIN TRANSACTION;

UPDATE konto **SET** saldo = saldo - 1000 **WHERE id** = 'A'; -- Abbuchung

UPDATE konto **SET** saldo = saldo + 1000 **WHERE id** = 'B'; -- Gutschrift

COMMIT; -- beides gilt | **ROLLBACK;** -- nichts gilt

- Klassiker: Überweisung – entweder *beide* Buchungen oder *keine*
- Bei Veggie Soles: „Lagerbestand senken **und** Bestellung anlegen **und** Rechnung schreiben“ – entweder ganz oder gar nicht

Schlägt mittendrin etwas fehl → ROLLBACK: die Datenbank ist wieder wie vorher.

ACID - die vier Garantien

	steht für	bedeutet
A	Atomicity	alles oder nichts – keine halben Transaktionen
C	Consistency	alle Regeln (Constraints) bleiben gültig
I	Isolation	parallele Transaktionen stören sich nicht
D	Durability	bestätigte Änderungen überleben auch einen Stromausfall

Ohne ACID kein verlässliches Online-Banking, kein E-Commerce, kein Ticketverkauf. Genau das unterscheidet ein echtes DBMS von einer Datei.

 **Selbst überlegen** – Notebook 21. Übung 4.1: für jede ACID-Eigenschaft

Cheat Card - Notebook 21

Begriff	Kurz
Tabelle / Zeile / Spalte	Relation / Datensatz (Tupel) / Attribut
Primärschlüssel (PK)	identifiziert jede Zeile – eindeutig, nie NULL
Fremdschlüssel (FK)	zeigt auf den PK einer anderen Tabelle
NULL	„kein Wert“ – prüfen mit IS NULL, $\neq 0$, $\neq$ " "
1:1 / 1:n / n:m	Beziehungstypen – n:m braucht eine Zwischentabelle
DECIMAL(p, s)	Datentyp für Geldbeträge
Transaktion / ACID	Operationen als Einheit / die vier Sicherheitsgarantien

Faustregel: erst das Datenmodell (Tabellen + Beziehungen), dann der Code.

Ausblick: ER-Diagramme und SQLite

- **Notebook 21b** – *ausführliche* Einführung ins **Entity-Relationship-Diagramm**: wie man ein Datenmodell *zeichnet*, Schritt für Schritt, bevor man eine einzige Tabelle anlegt.
- **Notebook 22** – SQLite mit Python: Tabellen anlegen, Daten einfügen, abfragen.
- **Notebook 23** – fortgeschrittene Abfragen: JOIN, GROUP BY, UPDATE/DELETE.

-- *Vorgeschmack auf NB 22:*

```
CREATE TABLE produkt (produkt_id INTEGER PRIMARY KEY, name TEXT,  
INSERT INTO produkt (name, preis) VALUES ('Eco-Sneaker', 89.95);
```

Heute gelernt

- ✓ Warum verknüpfte Tabellen Redundanz vermeiden
- ✓ Zeile / Spalte / Wert – die Anatomie einer Tabelle
- ✓ Primärschlüssel & NULL
- ✓ Fremdschlüssel + referentielle Integrität
- ✓ Beziehungstypen 1:1, 1:n, n:m
- ✓ SQL-Datentypen passend wählen
- ✓ Transaktionen & ACID

Zur Vertiefung im Notebook 21:

- Übungen 1.1-4.1 – Denkaufgaben mit Musterlösung
- Abschlussübungen Aufg. 1-4: vom PK/FK-Unterschied bis zum vollständigen Datenbankschema für ein Ticketsystem